

expectedGoals

February 8, 2026

```
[176]: # Straight from the book
import sys
assert sys.version_info >= (3, 5)

import pandas as pd

# Scikit-Learn 0.20 is required
import sklearn
assert sklearn.__version__ >= "0.20"

# Common imports
import numpy as np
import os

# To plot pretty figures
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.rc('axes', labelsize=14)
mpl.rc('xtick', labelsize=12)
mpl.rc('ytick', labelsize=12)

# Where to save the figures
PROJECT_ROOT_DIR = "."
CHAPTER_ID = "personal_project"
IMAGES_PATH = os.path.join(PROJECT_ROOT_DIR, "images", CHAPTER_ID)
os.makedirs(IMAGES_PATH, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = os.path.join(IMAGES_PATH, fig_id + "." + fig_extension)
    print("Saving figure", fig_id)
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
from zlib import crc32

def test_set_check(identifier, test_ratio):
```

```

        return crc32(np.int64(identifier)) & 0xffffffff < test_ratio * 2**32

def split_train_test_by_id(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: test_set_check(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]

```

1 Descriptive

Using pandas, we can start working on the dataset. First I want to group it by events:

```
[4]: df_games = pd.read_csv('./datasets/schedule/game_center/play_by_play/pbp2024205.
    ↪csv')
```

```
[5]: grouped = df_games.groupby("typeDescKey")
```

Which allows me to query any set of events at will using pandas' groups. Let's start with shots :

1.1 Shots

```
[6]: shots = grouped.get_group('shot-on-goal')
```

```
[175]: shots.columns
```

```
[175]: Index(['Unnamed: 0', 'eventId', 'timeInPeriod', 'timeRemaining',
            'situationCode', 'homeTeamDefendingSide', 'typeCode', 'typeDescKey',
            'sortOrder', 'periodDescriptor.number', 'periodDescriptor.periodType',
            'periodDescriptor.maxRegulationPeriods', 'details.eventOwnerTeamId',
            'details.xCoord', 'details.yCoord', 'details.zoneCode',
            'details.shootingPlayerId', 'details.shotType', 'details.goalieInNetId',
            'details.awaySOG', 'details.homeSOG', 'game_id'],
            dtype='object')
```

The history saving thread hit an unexpected error (OperationalError('attempt to write a readonly database')).History will not be written to the database.

1.1.1 Columns

Taking a look at the columns, I know I can : 1. Rank/order/group the plays according to any of these i.e players, coordinates, etc. 2. Filter/remove categories or events according to categories

Let's try to do each of these :

1.1.2 Shots grouped by player (1)

```
[8]: shots_by_player = shots.groupby("details.shootingPlayerId")
```

Which already gives us a lot of data which can be described and analyzed like the y-coordinates of a specific shooter.

```
[9]: shots_by_player['details.yCoord'].describe()
```

```
[9]:
```

	count	mean	std	min	25%	50%	\
details.shootingPlayerId							
8470600.0	764.0	5.492147	27.486705	-40.0	-22.0	9.0	
8470613.0	2316.0	3.837219	26.653408	-41.0	-21.0	11.0	
8470621.0	1110.0	-1.218919	16.669162	-41.0	-10.0	-1.0	
8471214.0	2654.0	0.130369	21.607705	-40.0	-21.0	-2.0	
8471215.0	1527.0	0.414538	15.398945	-36.0	-11.0	1.0	
...	...	...	...	...	...	...	
8484984.0	54.0	-11.277778	13.692605	-27.0	-22.0	-5.0	
8485105.0	53.0	-5.641509	29.142946	-33.0	-28.0	-4.0	
8485483.0	52.0	0.250000	20.525331	-20.0	-20.0	-1.0	
8485511.0	14.0	32.000000	0.000000	32.0	32.0	32.0	
8485512.0	27.0	11.518519	26.986279	-14.0	-14.0	-14.0	

	75%	max
details.shootingPlayerId		
8470600.0	31.75	40.0
8470613.0	28.00	41.0
8470621.0	6.00	41.0
8471214.0	21.00	40.0
8471215.0	11.00	34.0
...	...	...
8484984.0	4.00	7.0
8485105.0	39.00	39.0
8485483.0	19.25	23.0
8485511.0	32.00	32.0
8485512.0	39.00	39.0

[896 rows x 8 columns]

1.2 Slapshots only (2)

```
[10]: slap_shots = shots[shots['details.shotType'] == 'slap']
slap_shots.dropna(axis=1, how='all').describe()
```

```
[10]:
```

	Unnamed: 0	eventId	situationCode	typeCode	sortOrder	\
count	9.857500e+04	9.857500e+04	98575.000000	98575.0	98575.000000	
mean	2.830634e+06	6.899262e+03	1518.361907	506.0	420.307411	
std	1.632509e+06	1.123542e+05	128.179498	0.0	227.615215	
min	5.300000e+01	4.800000e+01	541.000000	506.0	11.000000	
25%	1.404860e+06	3.605000e+02	1541.000000	506.0	228.000000	
50%	2.844195e+06	6.670000e+02	1551.000000	506.0	422.000000	
75%	4.245779e+06	8.940000e+02	1551.000000	506.0	612.000000	
max	5.645568e+06	2.456180e+06	1560.000000	506.0	980.000000	

	periodDescriptor.number	periodDescriptor.maxRegulationPeriods	\
count	98575.000000	98575.0	
mean	2.025230	3.0	
std	0.813498	0.0	
min	1.000000	3.0	
25%	1.000000	3.0	
50%	2.000000	3.0	
75%	3.000000	3.0	
max	4.000000	3.0	

	details.eventOwnerTeamId	details.xCoord	details.yCoord	\
count	98575.000000	98575.00000	98575.000000	
mean	21.564991	0.65964	-0.357241	
std	16.088122	48.61840	22.866470	
min	1.000000	-99.00000	-41.000000	
25%	10.000000	-44.00000	-22.000000	
50%	18.000000	13.00000	0.000000	
75%	26.000000	45.00000	21.000000	
max	59.000000	98.00000	42.000000	

	details.shootingPlayerId	details.goalieInNetId	details.awaySOG	\
count	9.857500e+04	9.857500e+04	98575.000000	
mean	8.478391e+06	8.478503e+06	13.157717	
std	2.740834e+03	2.340953e+03	8.363504	
min	8.470600e+06	8.470594e+06	0.000000	
25%	8.476483e+06	8.476945e+06	6.000000	
50%	8.478136e+06	8.478492e+06	12.000000	
75%	8.480459e+06	8.480193e+06	19.000000	
max	8.485105e+06	8.484910e+06	49.000000	

	details.homeSOG	game_id
count	98575.000000	9.857500e+04
mean	13.892336	2.024021e+09
std	8.596866	3.800691e+02
min	0.000000	2.024020e+09
25%	7.000000	2.024020e+09
50%	13.000000	2.024021e+09
75%	20.000000	2.024021e+09
max	50.000000	2.024021e+09

1.3 Goals and shots-on-goal

```
[11]: shots = grouped.get_group('shot-on-goal')
goals = grouped.get_group('goal')
```

```
[12]: goals = pd.DataFrame(goals)
shots = pd.DataFrame(shots)
```

```
[13]: # Goals is one and shot is 0
goals['typeDescKey'] = 1
shots['typeDescKey'] = 0
```

```
[14]: goals = goals.dropna(axis=1, how="any")
shots = shots.dropna(axis=1, how="any")
```

```
[15]: shots_and_goals = pd.concat([shots, goals], ignore_index=True)
```

```
[16]: shots_and_goals.columns
```

```
[16]: Index(['Unnamed: 0', 'eventId', 'timeInPeriod', 'timeRemaining',
'situationCode', 'homeTeamDefendingSide', 'typeCode', 'typeDescKey',
'sortOrder', 'periodDescriptor.number', 'periodDescriptor.periodType',
'periodDescriptor.maxRegulationPeriods', 'details.eventOwnerTeamId',
'details.xCoord', 'details.yCoord', 'details.zoneCode',
'details.shootingPlayerId', 'details.shotType', 'details.goalieInNetId',
'details.awaySOG', 'details.homeSOG', 'game_id',
'details.scoringPlayerId', 'details.awayScore', 'details.homeScore'],
dtype='object')
```

```
[17]: shots_and_goals.describe()
```

```
[17]:
```

	Unnamed: 0	eventId	situationCode	typeCode	typeDescKey \
count	1.003380e+06	1.003380e+06	1.003380e+06	1.003380e+06	1.003380e+06
mean	2.822845e+06	6.595215e+03	1.516623e+03	5.058919e+02	1.080936e-01
std	1.630097e+06	1.105694e+05	1.457430e+02	3.104988e-01	3.104988e-01
min	5.000000e+00	2.000000e+00	1.010000e+02	5.050000e+02	0.000000e+00
25%	1.410442e+06	3.300000e+02	1.551000e+03	5.060000e+02	0.000000e+00
50%	2.823970e+06	6.330000e+02	1.551000e+03	5.060000e+02	0.000000e+00
75%	4.235137e+06	8.830000e+02	1.551000e+03	5.060000e+02	0.000000e+00
max	5.645829e+06	2.456291e+06	1.560000e+03	5.060000e+02	1.000000e+00

	sortOrder	periodDescriptor.number \
count	1.003380e+06	1.003380e+06
mean	4.185620e+02	2.037383e+00
std	2.338725e+02	8.605964e-01
min	1.000000e+01	1.000000e+00
25%	2.200000e+02	1.000000e+00
50%	4.150000e+02	2.000000e+00
75%	6.150000e+02	3.000000e+00
max	1.009000e+03	5.000000e+00

	periodDescriptor.maxRegulationPeriods	details.eventOwnerTeamId \
count	1003380.0	1.003380e+06
mean	3.0	2.028453e+01
std	0.0	1.551805e+01

min	3.0	1.000000e+00
25%	3.0	9.000000e+00
50%	3.0	1.800000e+01
75%	3.0	2.600000e+01
max	3.0	5.900000e+01

	details.xCoord	details.yCoord	details.shootingPlayerId \
count	1.003380e+06	1.003380e+06	8.949210e+05
mean	-3.763758e-01	-5.012996e-01	8.478865e+06
std	6.442992e+01	1.995913e+01	2.769198e+03
min	-9.900000e+01	-4.200000e+01	8.470600e+06
25%	-6.500000e+01	-1.500000e+01	8.476891e+06
50%	-1.000000e+00	0.000000e+00	8.478483e+06
75%	6.400000e+01	1.400000e+01	8.481028e+06
max	9.900000e+01	4.200000e+01	8.485512e+06

	details.goalieInNetId	details.awaySOG	details.homeSOG	game_id \
count	8.949210e+05	894921.000000	894921.000000	1.003380e+06
mean	8.478481e+06	12.962061	13.569295	2.024021e+09
std	2.364244e+03	8.475273	8.666391	3.803685e+02
min	8.470594e+06	0.000000	0.000000	2.024020e+09
25%	8.476932e+06	6.000000	6.000000	2.024020e+09
50%	8.478470e+06	12.000000	13.000000	2.024021e+09
75%	8.480193e+06	19.000000	20.000000	2.024021e+09
max	8.484910e+06	49.000000	51.000000	2.024021e+09

	details.scoringPlayerId	details.awayScore	details.homeScore
count	1.084590e+05	108459.000000	108459.000000
mean	8.478900e+06	1.883892	2.107912
std	2.773205e+03	1.493891	1.569967
min	8.470600e+06	0.000000	0.000000
25%	8.476921e+06	1.000000	1.000000
50%	8.478493e+06	2.000000	2.000000
75%	8.481186e+06	3.000000	3.000000
max	8.484984e+06	9.000000	8.000000

```
[37]: shots_and_goals_minimal = shots_and_goals[
      ["typeDescKey", "details.xCoord", "details.yCoord", "details.zoneCode", ]
      ]
```

```
[39]: print(shots_and_goals_minimal["details.zoneCode"].unique(),
      shots_and_goals_minimal["typeDescKey"].unique())
```

```
['O' 'D' 'N'] [0 1]
```

```
[40]: mapping = {'O': 1.0, 'D': 0.0, 'N': 0.5}
      shots_and_goals_minimal = shots_and_goals_minimal.copy()
```

```
shots_and_goals_minimal['zoneCode'] = shots_and_goals_minimal['details.
↳zoneCode'].map(mapping).astype(float)
```

So we can see shot-type is not given for goals. This is a big problem because we cannot predict how a goal is shot. We can still compute the important metrics : shot distance, shot angle and zone.

```
[41]: shots_and_goals_minimal["details.xCoord"].describe()
```

```
[41]: count    1.003380e+06
      mean     -3.763758e-01
      std      6.442992e+01
      min     -9.900000e+01
      25%     -6.500000e+01
      50%     -1.000000e+00
      75%      6.400000e+01
      max      9.900000e+01
      Name: details.xCoord, dtype: float64
```

```
[42]: x = shots_and_goals_minimal["details.xCoord"].astype(float)
      y = shots_and_goals_minimal["details.yCoord"].astype(float)
      shots_and_goals_minimal = shots_and_goals_minimal.copy()
      shots_and_goals_minimal['distance'] = np.sqrt(x**2 + y**2)

      # zoneCode == 1: distance = |distance - 50|
      mask1 = shots_and_goals_minimal['zoneCode'] == 1
      shots_and_goals_minimal.loc[mask1, 'distance'] = (shots_and_goals_minimal.
↳loc[mask1, 'details.xCoord']).abs()

      # zoneCode == 0 and distance <= 50: +50
      mask0 = (shots_and_goals_minimal['zoneCode'] == 0)
      shots_and_goals_minimal.loc[mask0, 'distance'] = -(shots_and_goals_minimal.
↳loc[mask0, 'distance']).abs()

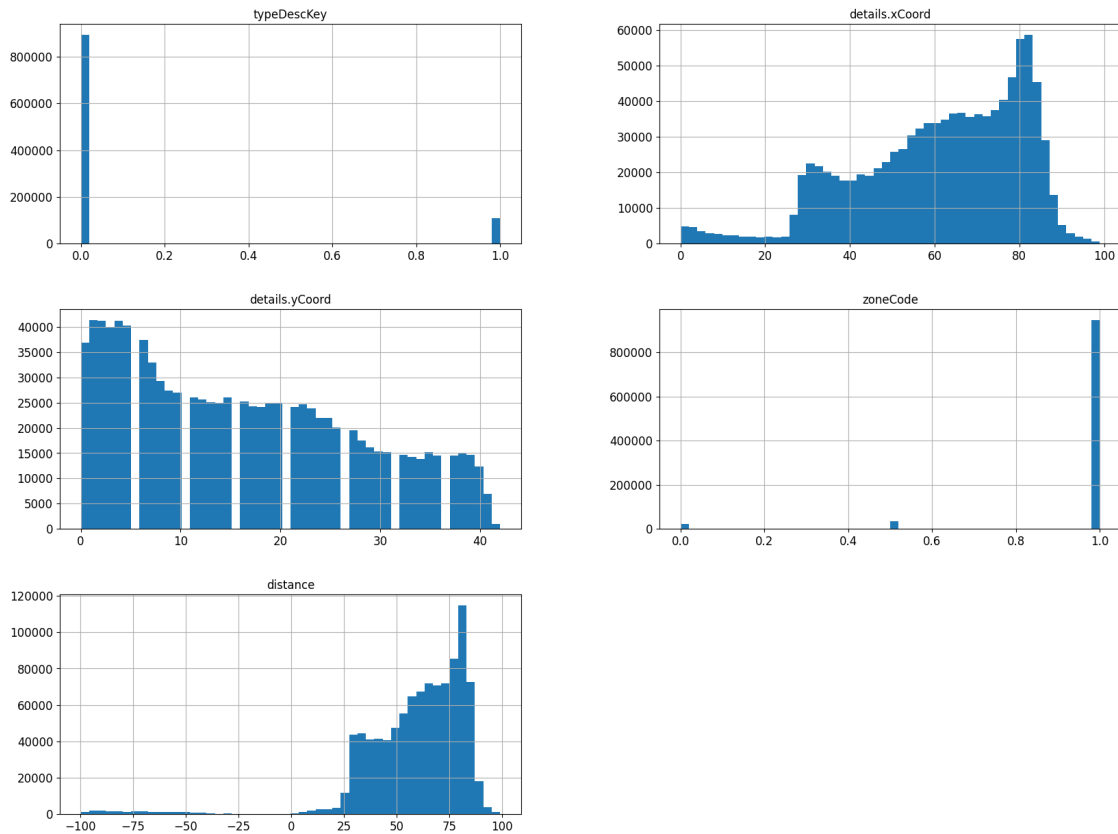
      # zoneCode == 1/2 pas traité
```

```
[117]: shots_and_goals_minimal['details.yCoord'] = pd.
↳to_numeric(shots_and_goals_minimal['details.yCoord'], errors='coerce')
shots_and_goals_minimal['details.yCoord'] = shots_and_goals_minimal['details.
↳yCoord'].abs()
shots_and_goals_minimal['details.xCoord'] = pd.
↳to_numeric(shots_and_goals_minimal['details.xCoord'], errors='coerce')
shots_and_goals_minimal['details.xCoord'] = shots_and_goals_minimal['details.
↳xCoord'].abs()
```

```
[118]: import matplotlib.pyplot as plt

shots_and_goals_minimal.hist(bins=50, figsize=(20, 15))
```

```
plt.show()
```



1.4 Machine learning part

```
[119]: hockey = shots_and_goals_minimal.copy()
np.random.seed(42)
```

```
[120]: from sklearn.model_selection import train_test_split
train_set, test_set = train_test_split(hockey, test_size=0.2, random_state=42)
```

```
[121]: train_set, test_set = train_test_split(hockey, test_size=0.2, random_state=42)
```

```
[122]: hockey["typeDescKey"].describe()
```

```
[122]: count      1.003380e+06
mean        1.080936e-01
std         3.104988e-01
min         0.000000e+00
25%         0.000000e+00
50%         0.000000e+00
75%         0.000000e+00
```

```
max      1.000000e+00  
Name: typeDescKey, dtype: float64
```

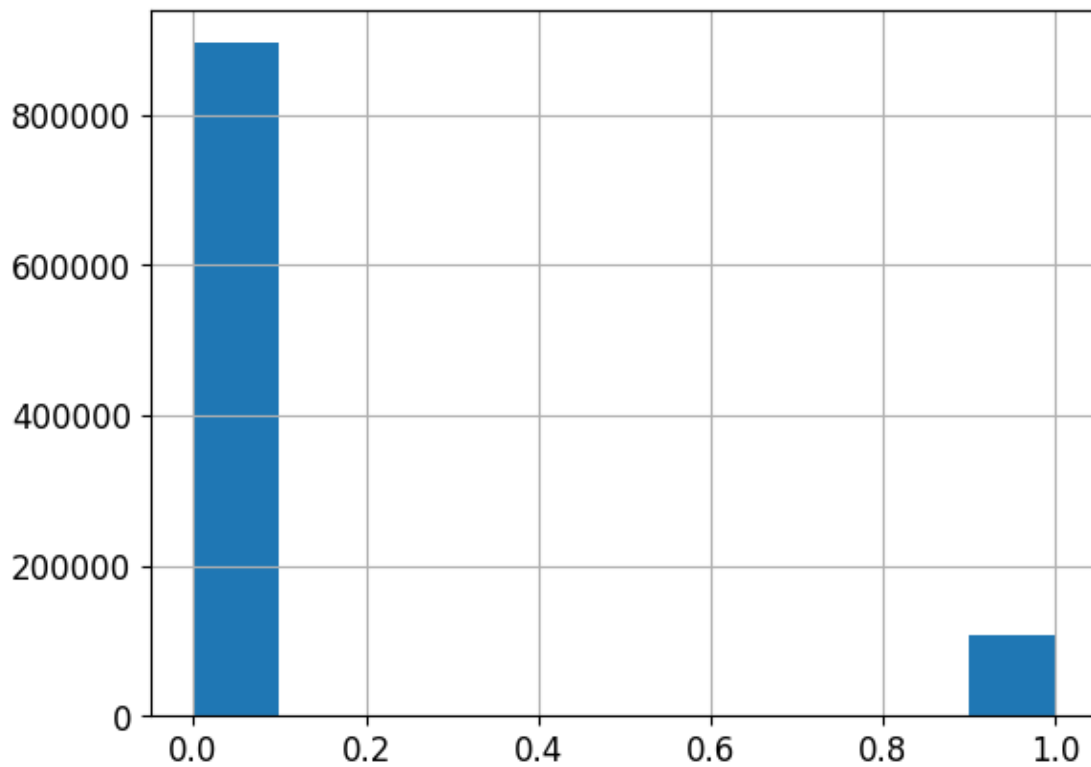
```
[123]: hockey['type_cat'] = pd.cut(  
        hockey['typeDescKey'],  
        bins=[-0.1, 0.5, 1.1],  
        labels=[0, 1]  
    )
```

```
[124]: hockey["type_cat"].value_counts()
```

```
[124]: type_cat  
0      894921  
1      108459  
Name: count, dtype: int64
```

```
[125]: hockey["type_cat"].hist()
```

```
[125]: <Axes: >
```



```
[126]: nan_rows = hockey[hockey["type_cat"].isna()]  
print(nan_rows)
```

```
hockey_no_nan = hockey.dropna(subset=["type_cat"])
hockey_no_nan
```

```

           typeDescKey  details.xCoord  details.yCoord  details.zoneCode  \
details.yCoord          NaN           NaN           NaN           NaN

```

```

           zoneCode  distance  type_cat
details.yCoord          NaN           NaN           NaN

```

```
[126]:
           typeDescKey  details.xCoord  details.yCoord  details.zoneCode  \
0                0.0            69.0            11.0                0
1                0.0            82.0             0.0                0
2                0.0            26.0            34.0                0
3                0.0            29.0            38.0                0
4                0.0            32.0             7.0                0
...
1003375          1.0            77.0            10.0                0
1003376          1.0            52.0            10.0                0
1003377          1.0            63.0            22.0                0
1003378          1.0            65.0            15.0                0
1003379          1.0            48.0            18.0                0

```

```

           zoneCode  distance  type_cat
0                1.0        69.0         0
1                1.0        82.0         0
2                1.0        26.0         0
3                1.0        29.0         0
4                1.0        32.0         0
...
1003375          1.0        77.0         1
1003376          1.0        52.0         1
1003377          1.0        63.0         1
1003378          1.0        65.0         1
1003379          1.0        48.0         1

```

```
[1003380 rows x 7 columns]
```

```
[127]: from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(hockey_no_nan,
↳hockey_no_nan["type_cat"]):
    strat_train_set = hockey_no_nan.loc[train_index]
    strat_test_set = hockey_no_nan.loc[test_index]
```

```
[128]: strat_test_set["type_cat"].value_counts() / len(strat_test_set)
```

```
[128]: type_cat
0    0.891905
1    0.108095
Name: count, dtype: float64
```

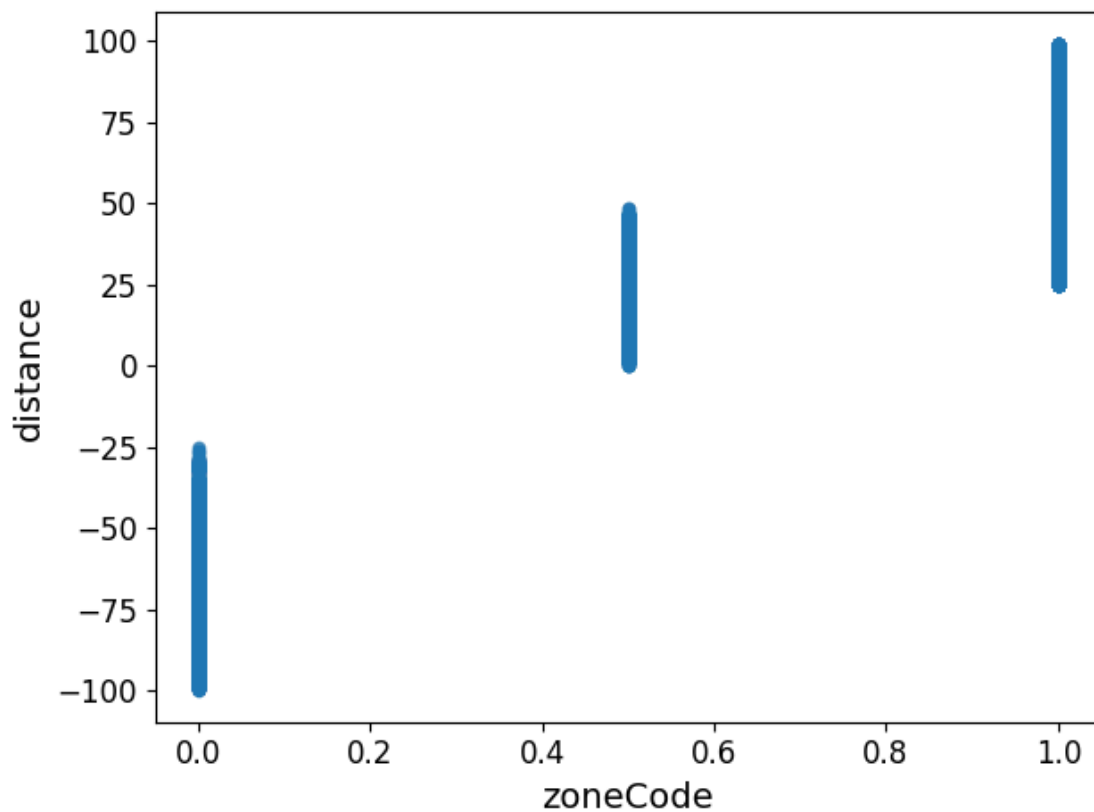
```
[129]: hockey["type_cat"].value_counts() / len(hockey)
```

```
[129]: type_cat
0    0.891905
1    0.108094
Name: count, dtype: float64
```

```
[130]: hockey = strat_train_set.copy()
```

```
[131]: hockey.plot(kind="scatter", x="zoneCode", y="distance", alpha =0.1)
save_fig("bad_visualization_plot")
```

Saving figure bad_visualization_plot



1.5 Correlations

```
[132]: # Only numeric columns
numeric_hockey = hockey.select_dtypes(include='number')
corr_matrix = numeric_hockey.corr()
corr_matrix["typeDescKey"].sort_values(ascending=False)
```

```
[132]: typeDescKey      1.000000
details.xCoord    0.148778
distance          0.118637
zoneCode          0.033168
details.yCoord   -0.189788
Name: typeDescKey, dtype: float64
```

Observing the correlation matrix for typeDescKey, we can see that y coordinates have the biggest (inverse) correlation, which is very telling : goals are usually scored from point blank.

1.6 Prepare data

```
[134]: hockey = strat_train_set.drop("typeDescKey", axis=1) # drop labels for training
      ↪ set

hockey_labels = strat_train_set["typeDescKey"].copy()
numeric_hockey = hockey.select_dtypes(include='number')

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")

imputer.fit(numeric_hockey)
imputer.statistics_
```

```
[134]: array([65., 15.,  1., 64.])
```

```
[135]: numeric_hockey.median().values
```

```
[135]: array([65., 15.,  1., 64.])
```

```
[136]: X = imputer.transform(numeric_hockey)
```

```
[137]: X
```

```
[137]: array([[84.,  7.,  1., 84.],
       [87.,  5.,  1., 87.],
       [81.,  5.,  1., 81.],
       ...,
       [84., 31.,  1., 84.],
       [61., 23.,  1., 61.]])
```

```
[41., 18., 1., 41.]], shape=(802704, 4))
```

```
[138]: hockey_tr = pd.DataFrame(X, columns=numeric_hockey.columns,  
                                index=hockey.index)
```

```
[139]: hockey_tr
```

```
[139]:
```

	details.xCoord	details.yCoord	zoneCode	distance
981063	84.0	7.0	1.0	84.0
153510	87.0	5.0	1.0	87.0
411045	81.0	5.0	1.0	81.0
254516	60.0	15.0	1.0	60.0
158893	82.0	8.0	1.0	82.0
...	...	...	...	...
340588	64.0	37.0	1.0	64.0
477249	58.0	31.0	1.0	58.0
171130	84.0	31.0	1.0	84.0
879438	61.0	23.0	1.0	61.0
703765	41.0	18.0	1.0	41.0

```
[802704 rows x 4 columns]
```

1.7 Training

```
[140]: from sklearn.linear_model import LogisticRegression  
  
log_reg = LogisticRegression(max_iter=10000)  
log_reg.fit(hockey_tr, hockey_labels)  
  
probs = log_reg.predict_proba(hockey_tr)
```

```
[141]: from sklearn.metrics import log_loss  
  
log_loss(hockey_labels, probs)
```

```
[141]: 0.31805372729617665
```

So we know that the mean of the cost function over the whole training set is about 0,32 which is reasonable, but not 0.

```
[167]: X = hockey_tr  
y = hockey_labels  
  
from sklearn.linear_model import Ridge  
ridge_reg = Ridge(alpha=10, solver="cholesky")  
ridge_reg.fit(X,y)
```

```
[167]: Ridge(alpha=10, solver='cholesky')
```

```
[168]: predictions = ridge_reg.predict(hockey_tr)
       predictions
```

```
[168]: array([0.18396396, 0.19748071, 0.18661117, ..., 0.08698015, 0.07764153,
        0.06161471], shape=(802704,))
```

```
[169]: hockey_labels.head()
```

```
[169]: 981063    1.0
       153510    0.0
       411045    0.0
       254516    0.0
       158893    0.0
       Name: typeDescKey, dtype: float64
```

We can see that the model is close for these 5 shots/goals, but that is not the point. We can say that according to the model, the first one is a bad goal (.184 against 1) and that the other shots were routine saves.

1.8 Conclusion

This very simple model can predict with reasonable error (0.318) the outcome of a given shot on goal taking only distance, coordinates, and zone parameters. Using this model, if I was working with a coach in the NHL, I would recommend maximizing point-blank range shots and to a lesser extent shots coming from the flanks.

To get a better model, it would be interesting to have shot types for goals, but the data is not available. Another way to better the model would be to look at the plays before the shot/goal. Some columns to add could be location of the last shot, time after faceoff, time after last pass, etc.

This would allow recommendations for plays leading to the shot which could be very helpful even if the log loss does not get that much better.